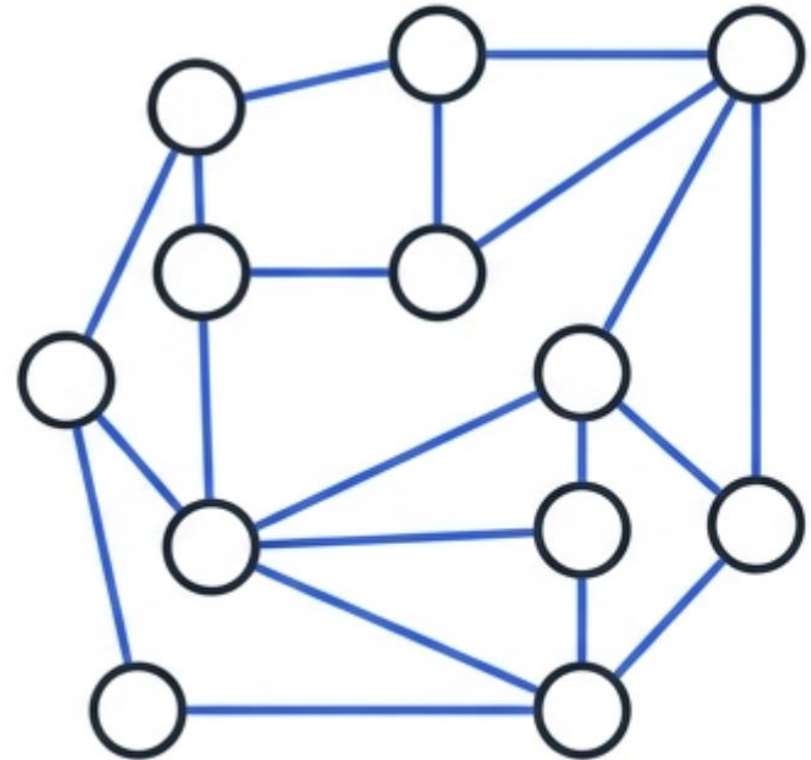


Grafos



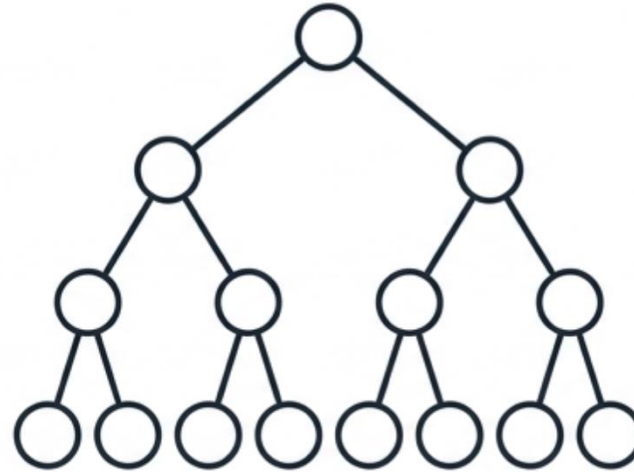
Objetivos

- Traducir el formalismo matemático de vértices y aristas a una interfaz polimórfica en C++ que garantice un contrato estricto de operaciones.
- Construir una **Matriz de Adyacencia** utilizando arreglos contiguous (`std::vector<std::vector<bool>>`), analizando su sobrecarga espacial de $\mathcal{O}(V^2)$
- Construir una **Lista de Adyacencia** optimizada en C++17 (`std::vector<std::vector<uint32_t>>`), prescindiendo de listas enlazadas tradicionales (`std::list`) para maximizar la localidad espacial.
- Contrastar algorítmicamente las operaciones de inserción de aristas, verificación de adyacencia y recuperación de vecindarios.

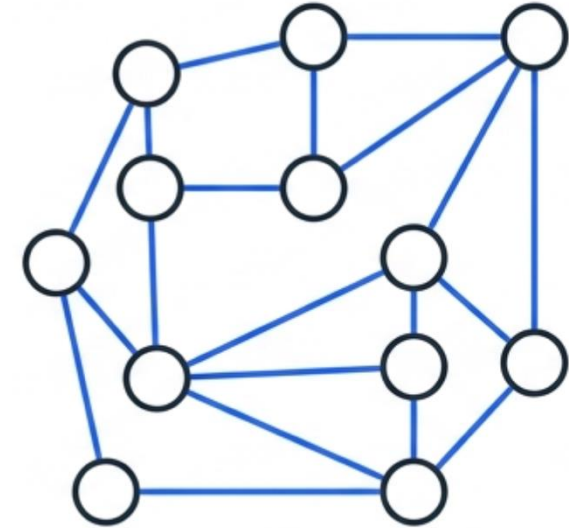
Relaciones arbitrarias



Datos Lineales



Datos Jerárquicos



Relaciones Arbitrarias

Las estructuras lineales y jerárquicas son insuficientes para representar redes cíclicas y asimétricas. El modelado de sistemas reales exige una abstracción superior.

Grafos



$$G = (V, E)$$

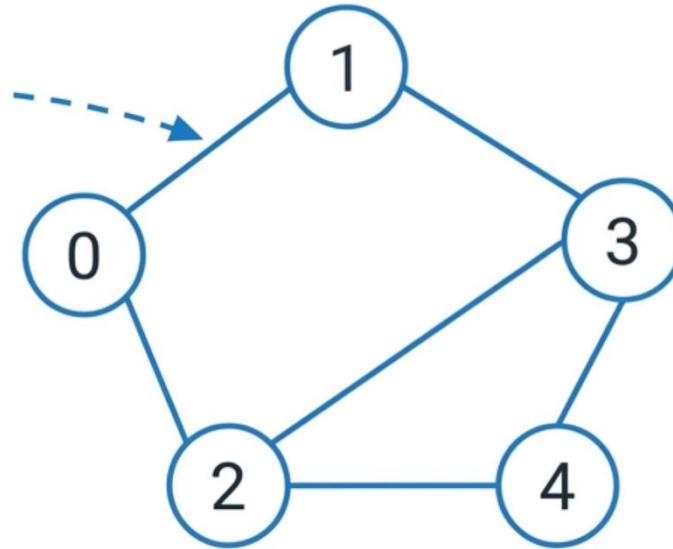
- V : Conjunto finito y no vacío de vértices (o nodos).
- V : Conjunto finito y no vacío de vértices (o nodos).
- $E \subseteq V \times V$: Conjunto de aristas (*edges*) que conectan pares de vértices.

Grafos

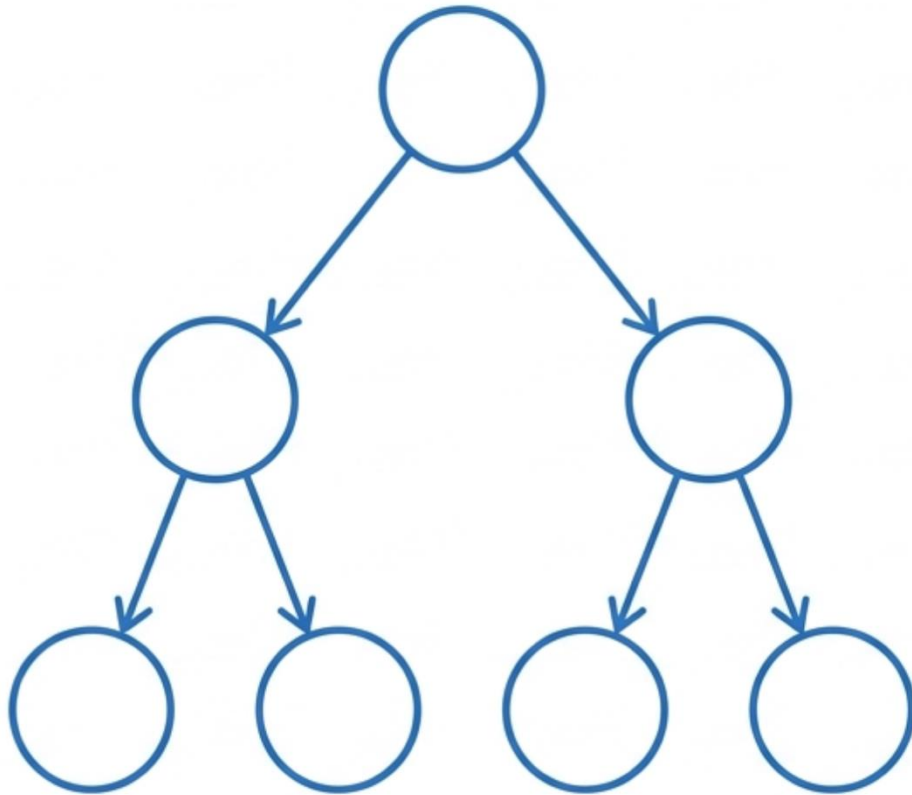
Cada par ordenado en el conjunto de aristas E se materializa como una conexión física en la topología del grafo G .

$$V = \{0, 1, 2, 3, 4\}$$

$$E = \{(0,1), (0,2), (1,3), (2,3), (2,4)\}$$

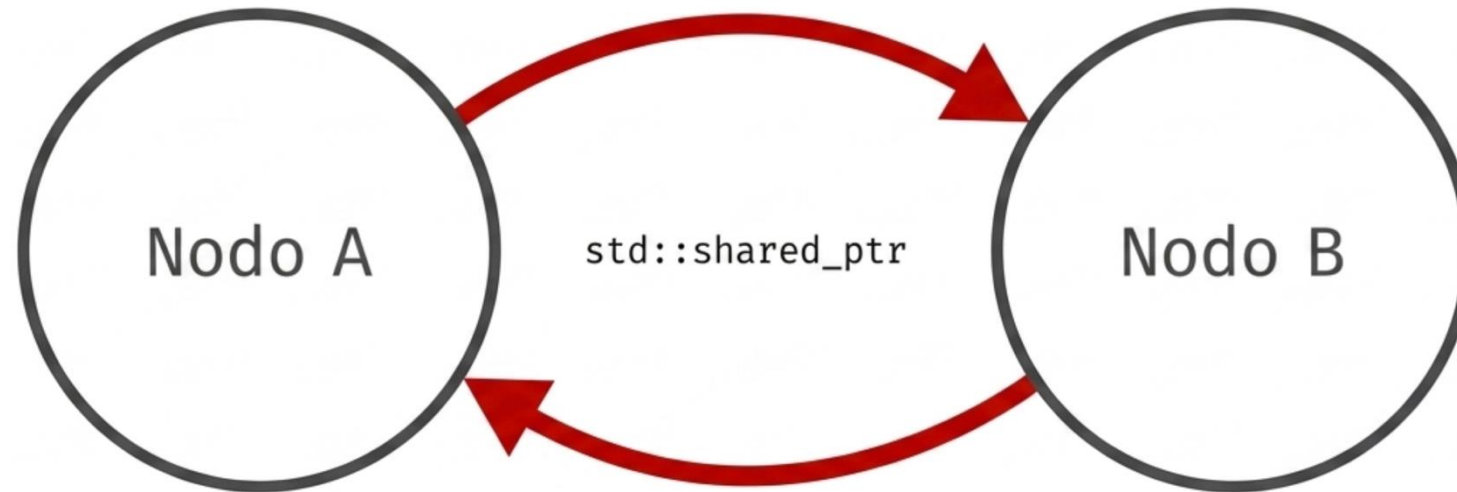


Estructura



En un árbol, un hijo tiene exactamente un padre. Esto nos permitió usar `std::unique_ptr` con éxito. Cuando el nodo padre es eliminado, su puntero se destruye y el hijo es liberado de la memoria automáticamente por C++.

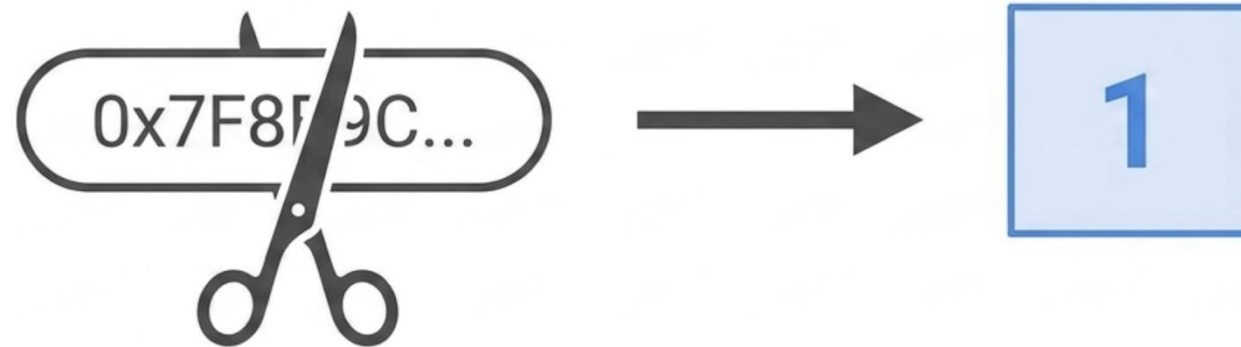
El problema de los ciclos



RESULTADO: FUGA DE MEMORIA (MEMORY LEAK) POR REFERENCIA CIRCULAR.

El cambio de paradigma

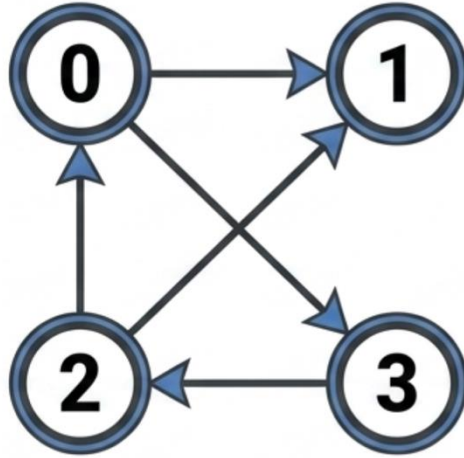
Para evitar que los nodos se bloqueen mutuamente, **la estructura de datos debe cambiar radicalmente.**



NUEVO PARADIGMA

En lugar de guardar el nodo completo en memoria mediante punteros (**shared_ptr**), simplemente guardaremos su número de identificación. Es decir, guardaremos un simple número entero (**uint32_t**).

La matriz de adyacencia



		DESTINATION VERTICES			
		0	1	2	3
SOURCE VERTICES	0	0	1	0	1
	1	0	0	0	0
	2	1	1	0	0
	3	0	0	1	0

CONCEPTO: UN ARREGLO BIDIMENSIONAL ($V \times V$)

Cada celda $A[i][j]$ indica si **existe una conexión** desde el vértice i hasta el vértice j .

Un valor de **1** (o el peso de la arista) significa que existe la conexión. Un **0** significa ausencia de conexión. Todo se reduce a coordenadas enteras.

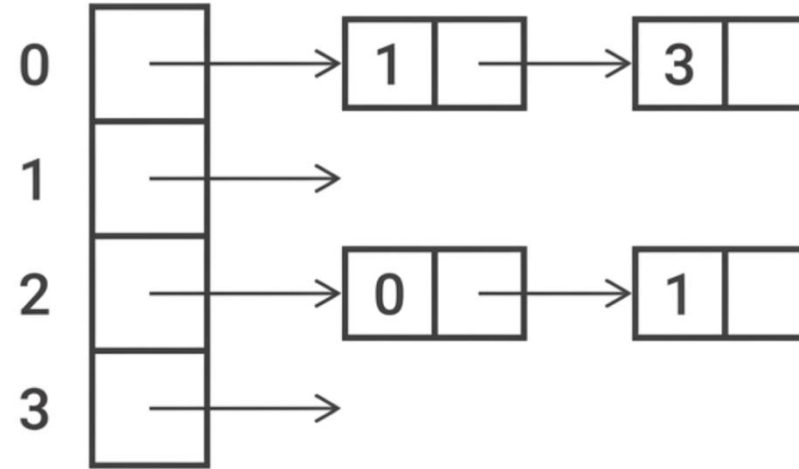
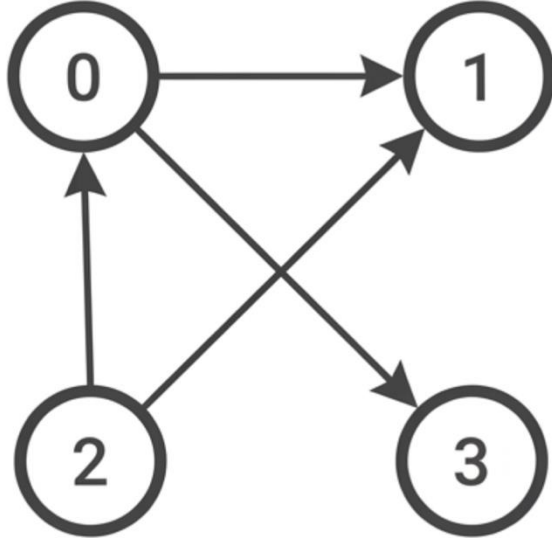
VENTAJA PRINCIPAL

Consultar si existe una arista específica es instantáneo ($O(1)$). Ideal para "grafos densos" donde casi todos los nodos se conectan entre sí.

DESVENTAJA PRINCIPAL

Desperdicio masivo de memoria si el grafo tiene muy pocas conexiones. Insertar un vértice nuevo exige redimensionar toda la estructura bidimensional.

Lista de adyacencia



CONCEPTO: ARREGLO PRINCIPAL CON ESTRUCTURAS SECUNDARIAS

Un arreglo principal donde cada índice es un Vértice. Cada posición almacena una lista dinámica (como `std::vector<int>`) que contiene únicamente los IDs numéricos de los vértices directamente conectados a él.

VENTAJA PRINCIPAL

Altamente eficiente en memoria. Excelente para “grafos dispersos”. Iterar sobre los vecinos directos de un nodo es extremadamente rápido. Es la implementación predominante en el mundo real.

DESVENTAJA PRINCIPAL

Verificar si una arista específica existe es más lento, ya que requiere barrer la sub-lista secuencialmente ($O(V)$ en el peor caso).

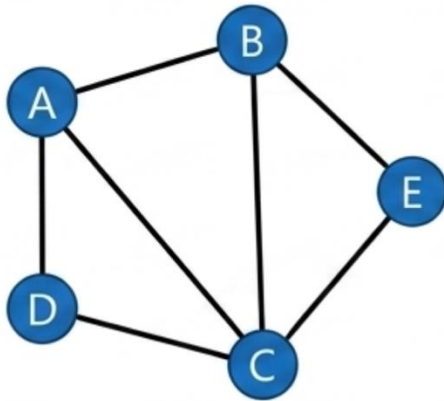
Comparación



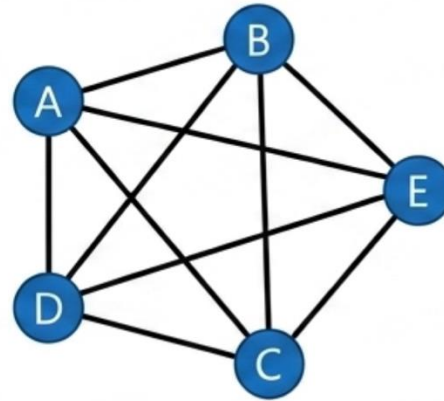
	MATRIZ DE ADYACENCIA	LISTA DE ADYACENCIA
Búsqueda de Arista	$O(1)$	$O(V)$ peor caso
Memoria Requerida	$O(V^2)$ (Alta)	$O(V + E)$ (Optimizada)
Adición de Vértice	Lenta $O(V^2)$	Inmediata $O(1)$
Iterar Vecinos	Ineficiente $O(V)$	Muy Eficiente

Densidad

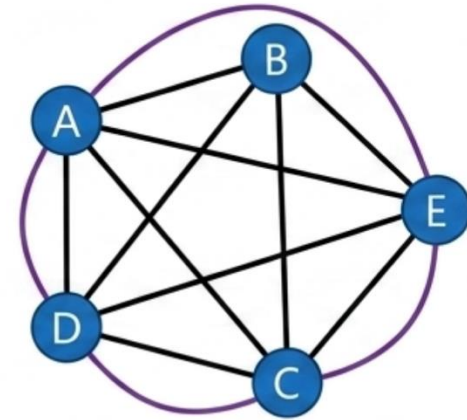
$$\rho = \frac{|E|}{|V|^2}$$



$|E| \ll |V|^2 \Rightarrow \rho \rightarrow 0.$
Aristas minúsculas frente
al máximo posible.

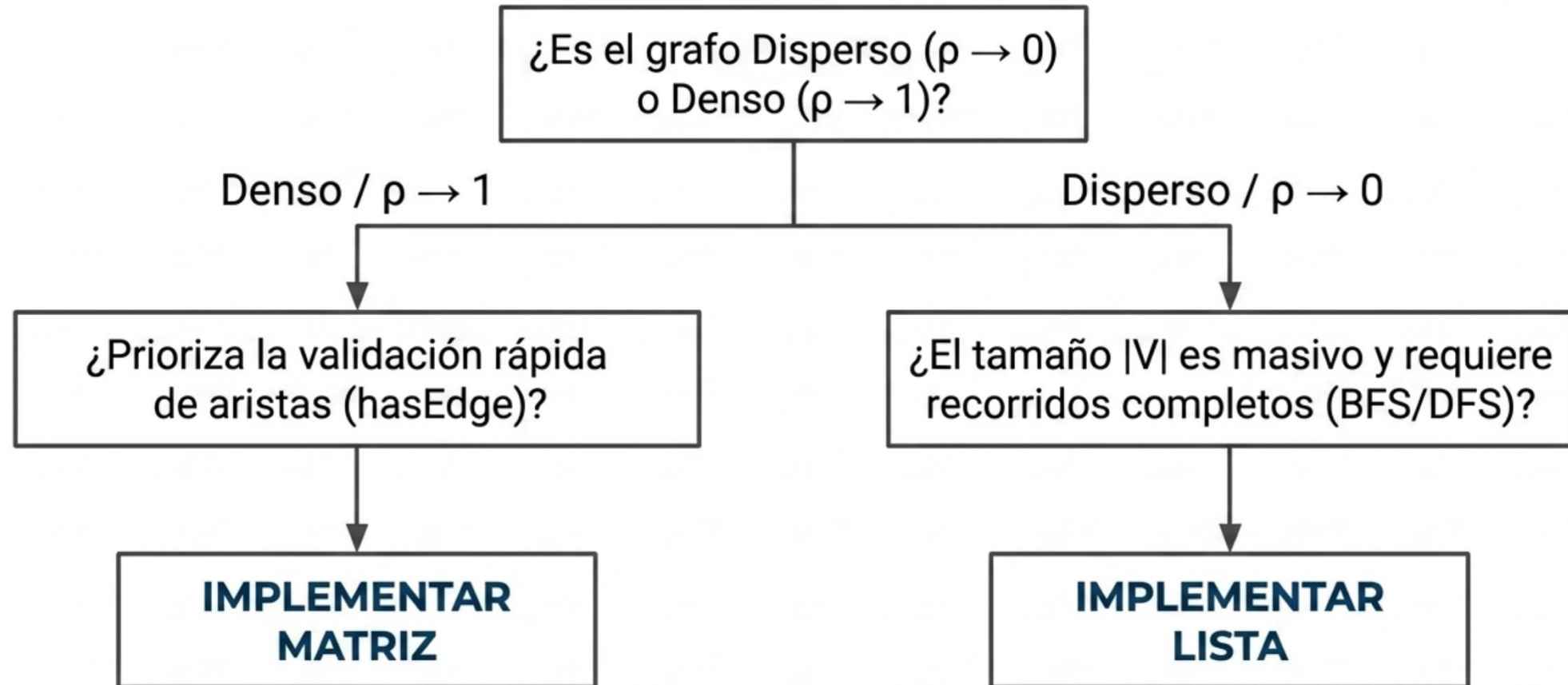


$|E| \approx |V|^2 \Rightarrow \rho \rightarrow 1.$



$|E| = \frac{|V|(|V|-1)}{2}.$
Peor caso asintótico de
conectividad.

Decisión arquitectónica





Rendimiento de un árbol AVL

La estricta gestión del Factor de Equilibrio garantiza matemáticamente que el árbol nunca degenerará en una estructura lineal, asegurando eficiencia logarítmica.

Operación	BST (Caso Promedio)	BST (Peor Caso)	Árbol AVL (Todos los Casos)
Búsqueda (find)	$O(\log n)$	$O(n)$	$O(\log n)$
Inserción (insert)	$O(\log n)$	$O(n)$	$O(\log n)$
Eliminación (remove)	$O(\log n)$	$O(n)$	$O(\log n)$

El costo adicional de las rotaciones $O(1)$ en memoria es trivial comparado con la prevención del peor caso $O(n)$.